

Project 11 (Bonus) — MCP Crew Multi-Agent Research Team Powered By MCP Servers

Title: MCP Crew Multi Agent Research Team Powered By MCP Servers

Type: Multi-agent (4 agents: 1 Supervisor + 3 specialists) · **Stack:** Python, LangGraph, `Command` tool, `mcp` (the official MCP Python SDK), OpenAI SDK, `python-dotenv`, `pydantic`, `asyncio` · **Level:** Advanced **Tagline:** A supervisor-led team of three specialists — Notes, Web, and Writer — where each specialist gets its own tools from its own dedicated MCP server, not from one shared Python toolbox.

Not part of the numbered Doc01-19 project arc — a bonus/portfolio project that combines two earlier bonus projects' ideas: [Project 7](#)'s MCP server-building and [Project 8](#)'s MCP client agent, applied inside [11_multi_agent_systems](#)'s Supervisor pattern. Read Project 8's README and Doc11's Core Concepts first if you haven't already — this file has the full build spec, not a pointer elsewhere.

Charter (what this project is)

Build a Supervisor-led team of 3 specialist agents, using the same `Command`-based routing from Project 4 and Doc11 — except this time, each specialist doesn't get its tools from a hardcoded Python list. Each specialist connects to its **own** MCP server, at startup, and discovers its own tools from there, the same way Project 8's single agent did. Project 4 proved a Supervisor can route between specialists that share one Python codebase. Project 8 proved one agent can get its tools from an MCP server instead of hardcoding them. This project asks what happens when you do both at once: a whole *team*, where each member's capabilities live behind its own protocol connection, not in one shared `tools.py`.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Picture a small research team again, like Project 4's ContentForge team. But this time, two of the three teammates don't keep their own notes and don't have their own web browser built in — they each have a phone line to a different outside service. One teammate's whole job is "call the notes service, ask it what's in there." Another teammate's whole job is "call the web service, ask it to fetch a page." Neither of them has that capability themselves — it lives on the other end of the phone line,

in a separate program neither of them wrote. The third teammate, the Writer, doesn't need a phone line at all — once the other two report back what they found, the Writer's whole job is turning that into one clear, readable report.

A manager — the Supervisor — decides who gets called for a given request, in what order, and when the team is actually done. That's MCP Crew: a **Notes Agent** that connects to its own MCP server for note storage and search, a **Web Agent** that connects to a different MCP server for fetching page content, and a **Writer Agent** that does neither — it's a plain LLM call that reads what the other two found and writes the final report. A **Command**-based Supervisor, built exactly the way Project 4's was, routes between all three.

Why build it this way, instead of giving one agent three MCP servers directly (which Project 8's Step 3 already showed you how to do)? Because in a real organization, different capabilities often really are owned by different teams, with different servers, different uptimes, and different owners — and splitting responsibility across specialist agents, each with a narrow job and its own dedicated connection, mirrors that reality better than one generalist agent juggling every connection itself. The Steps below build this up in the same order as every project in this curriculum: Step 1 proves each specialist works completely alone, MCP connection and all, before any teammate or Supervisor exists. Step 2 adds the Supervisor and proves it can route correctly between two already-working specialists. Step 3 adds the Writer, so the team's output becomes one coherent report instead of two separate specialists' raw findings sitting side by side. Step 4 asks the hardest, most realistic question: what does the team do when one specialist's MCP server just isn't there?

Before you read further — think about it yourself: if the Notes Agent and the Web Agent each connect to a *different* MCP server, does the Supervisor need to know anything about MCP at all, or can it just treat both specialists as ordinary nodes that happen to return a result? And if the Web Agent's server is down, should the Supervisor even try calling it, or should the Web Agent itself be the one to notice, fail cleanly, and report that upward? Sit with both of these before you read the Steps below.

What you're actually building, in one line: a Supervisor that routes work between three specialist agents, where two of them get their tools from their own separate MCP server instead of a shared toolbox.

Why this needs to exist: in a real company, different capabilities are often owned and run by different teams, with different servers and different uptimes — a system built the same way is more realistic, and safer, than one agent that depends on everything at once.

When you'd reach for this at a real job: when you're building an internal assistant that has to reach several genuinely different services owned by different teams, and you don't want one team's outage to quietly break every capability at once.

How it works, mechanically: each specialist opens its own connection to its own MCP server and discovers its own tools at startup; the Supervisor never touches those connections directly, it only calls each specialist as a node and reads back its finished result.

Why not just do it some simpler/different way: Project 8's Step 3 already showed one agent can hold three MCP servers directly, so why not always do that instead of building three separate agents? Because one agent juggling every connection means one bug, one slow server, or one bad prompt affects every capability at once — there's no clean boundary between them. Splitting into specialists means the Notes Agent's failure never touches the Web Agent's work, and each specialist's prompt only has to be good at one job instead of juggling three tool sets at once — which is also how real organizations actually split capabilities across teams.

Where This Fits

This is a bonus/portfolio project, not part of the main 5-project arc (Projects 1-5), and it isn't scored the way Project 6 tests architecture on a new domain. It's a deliberate combination of two other bonus projects' ideas, one level up: it assumes you understand **Project 7's** idea of an MCP server exposing its own tools, and **Project 8's** idea of a client agent discovering and calling tools from an MCP server at runtime, instead of hardcoding them. If you haven't built Project 7 or Project 8 yet, at least read their READMEs first — this project doesn't re-explain the MCP handshake, stdio transport, or the `ClientSession / stdio_client` shapes from scratch, since Project 8 already covers all of that in depth.

What's new here is putting that MCP-client idea inside Doc11's multi-agent territory: instead of one agent with two or three MCP servers (Project 8's Step 3), you now have *several agents*, each with its *own* MCP server, coordinated by a Supervisor using `Command` — exactly Project 4's routing pattern, just with MCP-sourced tools underneath two of the three specialists instead of hardcoded Python functions. This belongs in your portfolio as proof you can combine two real patterns — a standard tool protocol, and a coordinated multi-agent team — instead of only ever demonstrating them separately.

Setup (do this once, before Step 1)

```
cd project_11_mcpcrew
python -m venv .env
source .env/bin/activate          # Windows: .env\Scripts\activate
pip install mcp openai langchain langchain-openai langgraph python-dotenv pydantic
pip freeze > requirements.txt
```

Same `.env` setup as every other project:

```
OPENAI_API_KEY=sk-...
LOG_LEVEL=INFO
```

And the same `.env.example` with the key names but no real secret.

This project is `asyncio`-based wherever it talks to an MCP server, exactly like Project 8. If `async / await` is still unfamiliar, [08b_async_prereq's Core Concepts](#) is a quick, optional read — but Doc06's minimal client/server example, and Project 8's Step 1, already show the exact pattern (`asyncio.run(main())` around an `async def main()`) you'll be reusing here. You do **not** need Project 7's actual server running — Step 1 has you build a small, self-contained notes server of your own inside this folder, so this project doesn't depend on another one existing first.

A Real Example (so this isn't just theory)

Use this scenario, or one close to it, for every step below:

Scenario: MCP Crew is asked to put together a short status report on a made-up internal project, "Project Atlas." It has two sources: a **notes server** holding a handful of `.md`-style notes someone already wrote about Atlas, and a **web server** that can "fetch" a couple of known internal status pages (mocked — see Step 1).

Test these tasks (save them in `test_tasks.py` so you reuse the same ones every run): 1. "What do my notes say about Project Atlas?" — needs only the Notes Agent. 2. "Fetch the Project Atlas status page and summarize what it says." — needs only the Web Agent. 3. "Check my notes for anything about Project Atlas, fetch its status page too, and give me one combined report." — needs **both** specialists, then the Writer — this is the case that actually proves the Supervisor's multi-step routing and the Writer's synthesis both work, not just that each specialist works alone. 4. Task 3 again, but with the **Web server deliberately not started** — this is Step 4's test: the team should still produce a report from the Notes Agent's findings alone, with the report clearly saying the web status page couldn't be reached, not a crash and not a report that silently pretends the web check happened.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows you can combine two real, separately-useful patterns — a standard tool protocol (MCP) and a coordinated multi-agent team (Supervisor + `Command`) — instead of only ever demonstrating them in isolation.
- **Why it matters:** most real multi-agent systems at a company don't have all their tools living in one codebase — different teams own different services. A team where each specialist owns its own external connection is a much more realistic shape than one agent with every tool hardcoded in, or even one agent juggling every MCP server itself.

- **When you'd build something like this at a real job:** any internal "assistant team" where different capabilities are actually owned and run by different teams or services — a notes/wiki service, a web-fetch or search service, an internal ticketing system — each safely reachable only through its own protocol connection, coordinated by one routing layer.
- **How it's built:** a `Command`-based Supervisor (reused directly from Project 4/Doc11), two specialist agents that each open their own MCP client connection at call time (reusing Project 8's connection and tool-bridge patterns), a Writer agent that's a plain LLM call with no tools, and a degradation path so one specialist's dead server doesn't take down the whole team.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The Supervisor tries to call a specialist's tools directly, instead of letting the specialist own its own MCP connection	Keep the boundary clean: the Supervisor only ever calls a specialist <i>node function</i> and reads its result from shared state — it never touches <code>ClientSession</code> or <code>stdio_client</code> itself, exactly like Project 4's Supervisor never called a tool directly, only routed to the agent that owns it
The Writer's report reads like two disconnected paragraphs stapled together	Give the Writer's prompt both specialists' findings <i>and</i> an explicit instruction to synthesize, not concatenate — the same lesson Doc11's "Generator" pattern teaches: a plain LLM call still needs a deliberately written prompt, not just "here's some text, summarize it"
A specialist's MCP connection opens and closes correctly in Step 1's standalone test, but hangs once it's a graph node	Check whether your graph node is <code>async</code> and being invoked with <code>.ainvoke()</code> / <code>astream()</code> , or whether you're wrapping the same <code>async def</code> in <code>asyncio.run()</code> inside a plain sync node — pick one pattern and use it consistently across every specialist node
The Web Agent's server is down, and the whole graph crashes instead of degrading	Reuse Project 8 Step 4's exact instinct: wrap the connection attempt in <code>asyncio.wait_for(...)</code> inside <code>try / except</code> , catch it <i>inside</i> the Web Agent's own node function, and write a clear "unavailable" result into shared state instead of letting the exception escape the node
The final report doesn't mention that a specialist's data is missing, even though Step 4's degradation worked correctly	The degradation flag has to actually reach the Writer's prompt — check your shared state includes something like <code>web_unavailable: bool</code> , and that the Writer's prompt-building code reads it and says so explicitly, instead of just silently having an empty <code>web_findings</code> field

Built During These Documents

[06_tools_function_calling](#) → [11_multi_agent_systems](#)

Plan Before You Code

Same process as every project (see [15_five_projects_index](#)): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — before writing any connection code. Before you open your editor, sketch on paper which pieces of Project 4's Supervisor you're reusing unchanged, and which pieces of Project 8's MCP client code you're reusing unchanged — most of this project is combining two things you've already built correctly once, not inventing new patterns from scratch.

How To Build This — Step by Step

Step 1 — Two Standalone MCP-Connected Specialists

Project: MCP Crew Multi-Agent Research Team Powered By MCP Servers — Step 1 of 4: Two Standalone MCP-Connected Specialists

What this step does: builds the Notes Agent and the Web Agent, each connecting to its own small MCP server, each one proven to work completely alone — no Supervisor, no LangGraph, no teammate exists yet. **Why this step matters:** this is the same "prove the connection before adding complexity" instinct every project in this curriculum starts with. If a specialist's MCP connection is broken, debugging that *inside* Step 2's Supervisor graph means debugging two things at once — the connection and the routing — instead of one at a time. **When you'll hit this for real:** the first time you wire any external service into a multi-agent system — you always prove the one connection works standalone before trusting a coordinator to call it correctly. **Read first:** [06_tools_function_calling Core Concepts — "MCP servers and clients, and how they actually connect"](#), [06_tools_function_calling Core Concepts — "MCP's 3 building blocks: Tools, Resources, and Prompts"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `notes_server.py` : a `FastMCP` server (a simplified version of Project 7's `MCPForge` — you don't need `SQLite` or a `Resource/Prompt` here, just the two `Tools`) exposing `add_note(text: str) -> str` and `search_notes(query: str) -> list[str]`, backed by a plain in-memory list, seeded with 3-4 made-up notes at startup so `search_notes` has something real to find. 2. Write `web_server.py` : a `FastMCP` server exposing exactly one tool, `fetch_page(url: str) -> str`. Make this a **mocked, stubbed fetch** — a small Python dict mapping 2-3 known URLs (like `"https://intranet.example.com/atlas/status"`) to canned page text, and a clear `"No content available for this URL."` string for anything else. Write a comment at the top of this file saying plainly that a real deployment would connect this tool to a real fetch/search MCP server (or the public reference fetch server) instead of a hardcoded dict — this project mocks it on purpose, for reliable, repeatable teaching. 3. Write `notes_agent.py` : `run_notes_agent(task: str) -> str`, which opens its own connection to `notes_server.py` (the same `stdio_client / ClientSession / initialize()` shape from Project 8's Step 1), discovers its tools, and runs a small ReAct-style loop (reuse Project 8's Step 2 pattern) to answer the task using `search_notes` and/or `add_note`. 4. Write `web_agent.py` : `run_web_agent(task:`

`str`) -> `str`, the same shape, connecting to `web_server.py` and using `fetch_page`. 5. Write `main.py` that calls `run_notes_agent` and `run_web_agent` one after another, on tasks 1 and 2 from **A Real Example**, and prints both results — confirm each specialist works completely on its own before Step 2 puts a Supervisor in front of them.

Your files after Step 1:

```
project_11_mcpcrew_multi_agent_mcp/
├── .env / .env.example
├── config.py                (reused pattern from Doc01)
├── logging_setup.py         (reused pattern from Doc01)
├── notes_server.py          → FastMCP: add_note(text), search_notes(query) – s
├── web_server.py            → FastMCP: fetch_page(url) – mocked/stubbed conte
├── mcp_connection.py        → connect_stdio_server(command, args) -> Client
├── notes_agent.py           → run_notes_agent(task) -> str, standalone, c
├── web_agent.py             → run_web_agent(task) -> str, standalone, co
└── main.py                  → runs both specialists standalone, no sup
```

Step 2 — A Supervisor Routes Between Them

Project: MCP-Crew-Multi-Agent-Research-Team-Powered-By-MCP-Servers — Step 2 of 4: A Supervisor Routes Between Them

What this step does: wraps Step 1's two proven specialists in a real LangGraph graph, and adds a `Command`-based Supervisor (Project 4's exact pattern) that decides, per request, whether to call the Notes Agent, the Web Agent, or both, in sequence. No Writer yet — whichever specialist(s) ran, their raw result is the graph's output. **Why this step matters:** this is the actual multi-agent part of the project — proving a Supervisor can route correctly to specialists whose tools it knows nothing about, since each specialist's MCP connection is fully private to that specialist's own node. **What's new vs. Step 1:** a shared `TeamState`, a Supervisor node using `Command`, and Step 1's two specialists rewired as graph nodes instead of functions called directly by `main.py`. **What stays the same:** the specialists' own internal logic — the MCP connection, tool discovery, and ReAct loop inside `notes_agent.py` and `web_agent.py` — is unchanged from Step 1. You're wrapping them, not rewriting them. **When you'll hit this for real:** any time a working single-purpose agent needs to join a bigger, coordinated system — the same moment Project 4's Steps 1-4 each individually proved specialists before Step 5 added the Supervisor. **Read first:** [11_multi_agent_systems Core Concepts — "The `Command` tool: today's way of handing work between agents"](#), [11_multi_agent_systems Core Concepts — "Shared state design: what goes in the shared state, and what doesn't"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Design `state.py`: a `TeamState(TypedDict)` with `task: str`, `notes_findings: str`, `web_findings: str`, and routing info like `next_agent: str`. Follow Doc11's rule directly — only each specialist's *finished* result goes in shared state; each specialist's

own scratchpad (its ReAct loop's intermediate steps) stays private inside that specialist's own node function. 2. Rewrite `notes_agent.py` and `web_agent.py` to expose a graph-node function too — `notes_agent_node(state: TeamState) -> Command`, `web_agent_node(state: TeamState) -> Command` — each one calls its existing Step 1 function internally, writes the result into the right shared-state field, and returns a `Command` sending control back to the Supervisor. 3. Write `supervisor.py`: a node that looks at `state["task"]` (and what's already been filled in) and returns a `Command` routing to `"notes_agent"`, `"web_agent"`, or `"FINISH"` — for task 3 from **A Real Example**, this means routing to one specialist, seeing its result land in state, then routing to the other, before finishing. 4. Write `graph.py`: a `StateGraph(TeamState)` wiring the Supervisor and both specialist nodes together, with the Supervisor as the entry point. 5. Update `main.py` to build and run the graph on tasks 1, 2, and 3 from **A Real Example** — confirm task 3's final state has *both* `notes_findings` and `web_findings` filled in, proving the Supervisor really called both specialists in one run, not just the first one it happened to pick.

Your files after Step 2:

```
project_11_mcpcrew_multi_agent_mcp/
├─ config.py
├─ logging_setup.py
├─ notes_server.py
├─ web_server.py
├─ mcp_connection.py
├─ state.py                → TeamState(TypedDict): task, notes_findings, web_fin
├─ supervisor.py           → Command-based router: notes_agent / web_agent / F
├─ notes_agent.py          → adds notes_agent_node(state) -> Command, wraps
├─ web_agent.py            → adds web_agent_node(state) -> Command, wraps
├─ graph.py                → StateGraph wiring supervisor + both special
└─ main.py                 → runs the graph, prints whichever specialis
```

Step 3 — The Writer Agent Synthesizes a Final Report

Project: MCP-Crew-Multi-Agent-Research-Team-Powered-By-MCP-Servers — Step 3 of 4: The Writer Agent Synthesizes a Final Report

What this step does: adds the Writer — a plain LLM call, no MCP server, no tools at all — which the Supervisor calls last, once the specialists it needed have already contributed. It turns "here's what 2 agents found separately" into one coherent report. **Why this step matters:** two specialists' raw findings sitting side by side in state isn't a report — it's two paragraphs a person still has to stitch together themselves. The Writer is the piece that actually produces the thing the user asked for.

What's new vs. Step 2: a Writer agent and node; the Supervisor's routing now ends with a call to the Writer instead of `FINISH` directly. **What stays the same:** the Notes Agent's and Web Agent's own logic — untouched since Step 1, still called the same way through their Step 2 node wrappers.

When you'll hit this for real: any pipeline where multiple sources feed into one final deliverable — a report, a summary, an answer — that a person or downstream system actually reads; raw findings

from separate sources almost always need a synthesis step, not just concatenation. **Read first:** [11_multi_agent_systems Core Concepts](#) — "Every pattern below: what / why / when / trade-off" (the "Generator → Critic → Revision" entry, for the synthesis-role idea), [05_langchain_fundamentals Core Concepts](#) — "LCEL: connecting pieces with `|`" (the Writer is a plain chain, same shape as Project 4's Step 1 Writer).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `writer_agent.py`: `run_writer(notes_findings: str, web_findings: str) -> str`, a plain `prompt | ChatOpenAI | output_parser` chain (Doc01's config/logging, no tools, no MCP connection at all). Write the prompt to explicitly instruct synthesis, not concatenation — something like "combine these findings into one short report; don't just restate each source separately." 2. Add `writer_agent_node(state: TeamState) -> Command`, which reads `notes_findings` and `web_findings` from state, calls `run_writer`, writes the result into a new `report: str` field, and returns a `Command` routing to `END`. 3. Update `supervisor.py` so that once every specialist the task actually needed has contributed (check your routing logic from Step 2 against `state["next_agent"]` or a small `completed` tracking field), it routes to `"writer_agent"` instead of straight to `"FINISH"`. 4. Update `graph.py` to add the Writer node, and update `main.py` to run all 4 tasks from **A Real Example** except task 4 (that's Step 4's test) — confirm task 3 produces one combined report mentioning both the notes content and the fetched page content, not two disconnected paragraphs.

Your files after Step 3:

```
project_11_mcpcrew_multi_agent_mcp/
├─ config.py
├─ logging_setup.py
├─ notes_server.py
├─ web_server.py
├─ mcp_connection.py
├─ state.py                → adds report: str
├─ supervisor.py           → now routes to writer_agent once specialists are done
├─ notes_agent.py
├─ web_agent.py
├─ writer_agent.py         → run_writer(notes_findings, web_findings) -> str
├─ graph.py               → adds the writer node
└─ main.py                → runs the full team, prints the final synthesis
```

Step 4 — Graceful Degradation When One MCP Server Is Down

Project: MCP Crew Multi-Agent Research Team Powered By MCP Servers — Step 4 of 4: Graceful Degradation When One MCP Server Is Down

What this step does: makes the whole team survive the Web Agent's MCP server being unreachable — the Supervisor should still produce a report using just the Notes Agent's findings,

with the report clearly saying the web check couldn't be done, instead of the whole run crashing.

Why this step matters: this is Project 8 Step 4's exact lesson — a dead MCP server shouldn't crash everything depending on it — proven again one level up, at the team level instead of the single-agent level. A team is less trustworthy than a single agent if it's *more* fragile, not less, when one piece goes down. **What's new vs. Step 3:** a timeout + `try / except` around the Web Agent's connection attempt, a `web_unavailable: bool` field in shared state, and a Writer prompt that reads that flag and says so in the report. **What stays the same:** the Notes Agent, and the Web Agent's behavior when its server *is* reachable — resilience wraps around the existing team, it doesn't rewrite it. **When you'll hit this for real:** the first time you demo this project (or any multi-agent system with external connections) to someone else and one of your servers isn't running yet — which will happen, and is exactly why this step exists before you call the project done. **Read first:** [07_ai_agents Core Concepts — "Common single-agent failures, worth naming now"](#), [11_multi_agent_systems Core Concepts — "Error propagation between agents"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. In `mcp_connection.py`, wrap the connect-plus- `initialize()` call in `asyncio.wait_for(..., timeout=...)`. In `web_agent.py`'s `run_web_agent`, catch a timeout or connection failure with `try / except`, log it clearly (`"[web_agent] server unreachable: <reason>, continuing without it"`), and return a clear sentinel string (like `"WEB_UNAVAILABLE"`) instead of letting the exception escape. 2. Update `web_agent_node` so that when it sees that sentinel, it sets `state["web_unavailable"] = True` and writes an empty or clearly-labeled `web_findings`, instead of crashing the node. 3. Update `supervisor.py` so it still routes to `writer_agent` after a failed Web Agent call — a missing web server is not a reason to end the run early, it's a reason to finish with less information. 4. Update `writer_agent.py`'s prompt-building so that when `web_unavailable` is `True`, the prompt tells the model plainly that the web check could not be completed, and instructs it to say so in the report rather than silently omitting it or inventing something. 5. Write `test_degraded_run.py`: run task 3 from **A Real Example** twice — once with both servers up (confirm a full report using both sources), once with `web_server.py` deliberately never started (confirm the run still finishes, using only the Notes Agent's findings, and the final report clearly states the web page couldn't be reached).

Your files after Step 4 (final):

```
project_11_mcpcrew_multi_agent_mcp/
├─ config.py
├─ logging_setup.py
├─ notes_server.py
├─ web_server.py
├─ mcp_connection.py      → connect wrapped in a timeout + try/except; f
├─ state.py              → adds web_unavailable: bool
├─ supervisor.py         → still routes to writer_agent after a fai
├─ notes_agent.py
├─ web_agent.py          → catches an unreachable server, sets w
├─ writer_agent.py       → prompt explicitly flags missing we
```

```
├─ graph.py
├─ main.py
└─ test_degraded_run.py
```

→ full demo: runs with both servers
→ confirms the team finishes and reports

Final Deliverable: MCP Crew-Multi-Agent-Research-Team-Powered-By-MCP-Servers — a Supervisor-led team of three agents, where the Notes Agent and Web Agent each get their tools from their own dedicated MCP server instead of hardcoded Python, the Writer synthesizes both specialists' findings into one coherent report, and the whole team keeps working — with a clear, honest gap noted — when one specialist's MCP server is unreachable.

Checklist Before You Call This Done

- [] The Notes Agent and Web Agent each connect to their own separate MCP server — no tool is hardcoded directly into either specialist's own Python code
- [] Each specialist was proven working completely alone (Step 1) before the Supervisor (Step 2) ever routed to it
- [] The Supervisor routes using the `Command` tool, the same pattern as Project 4 — not a hand-made text-code router
- [] A test where the Supervisor calls both specialists in one run, and the Writer's final report genuinely combines both, not just concatenates them
- [] Shared state only holds each specialist's *finished* result, not its private scratchpad or tool-call history
- [] A test where the Web Agent's MCP server is deliberately not running — the team still finishes, using only the Notes Agent's findings, and the final report clearly says the web check couldn't be done
- [] You can explain, for each specialist, exactly which piece of code owns its MCP connection, and confirm the Supervisor never touches that connection directly

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**